

AFG Prover

AFG Prover

Introduction

Welcome to AFG Prover - a growing and expanding interactive and automated theorem prover!

Aim of this project is to create a simple but working interactive theorem prover for first order logic, induction, set theory and modal logic, as well as a basic automated theorem prover for first order logic. Most of this project's ideas and values are take as inspiration from provers/frameworks such as Vampire, Isabelle-HOL, Coq, Lean ...

As of now (v 0.1) only an interactive theorem prover for FOL has been implemented.

Keep in mind that this is a one man passion project done mostly for boosting my resume.

Underlining Structure

Underneath the AFG prover lies the AIS language. An interpreted made-up language that can process the user input into Formula tokens that are then parsed into Sequents. Full documentation for the AIS language is given below.

Every formula presented in AFG prover is considered a "Sequent"; comprised of its assumption part and its goal part. Each part being a formula itself. Although a canonical approach with the use of currying is

more pragmatic, I wanted to make my own system, so I ended up using a "Sequent style approach". (One must keep in mind that the use of word 'sequent' is rather loose here)

The main engine of the AFG prover is its set of different interactive provers, each based on different logics or principles. Implemented provers thus far are:

1. Natural Deduction

- Aristotle (Intuitionistical logic, Propositional logic)
- Frege (Intuitionistical logic, Propositional logic, First order logic)
- Godel (Intuitionistical logic, Propositional logic, First order logic, Induction) (In progress)
- Russel (Intuitionistical logic, Propositional logic, First order logic, Induction, Set theory) (In progress)

2. Sequent Calculus (Planned)

3. Modal logic

- Kripke (In progress)

All of them are equipped with the same parser, but they interpret the parsed text differently.

The program comes with a built in text-editor that can open and read .ais files

AIS Syntax

Goals to be proven are given as "sequents" in AFG. They are of the following style:

$$\text{assumption1, assumption2} \dots, \text{assumptionN} \vdash \text{goal}$$

where each assumption I and goal is a Formula. Note however that the turnstile symbol ($\vdash$) here is interpreted as "to-be-proven" rather than its classical interpretation of "proves"!

Every tactic/rule is applied only to the first assumption in the list of assumption (assumption1), except for tactic *axiom* (*assumption*) (see below), in the first sequent (see tactic *next* below).

Whenever the current goal is proven, it is removed from the list of current goals. If there are no more goals to be proven, the phrase "Done." is printed on the screen.

Each statement to be proven in AFG starts with *lemma* keyword. There are two variants of this keyword, with and without the colon symbol (:).

If there is no colon symbol the following structure is needed:

- keyword *lemma* followed by zero or more assumption formulas divided by (;) symbol, followed by a turnstile symbol ($\vdash$) followed by one and only one goal formula.

Example:

```
lemma p; p => q | - q
```

If however, there is a colon symbol following the *lemma* keyword then assumptions and goal must be split into new lines. Keyword *assume* is needed to indicate that the following formula is an assumption, while the keyword *show* is needed to indicate that the following formula is the goal. The order of these combinations of keywords - formulas doesn't matter

Example:

```
lemma:  
assume p  
show q  
assume p => q
```

Below are listed currently implemented tactics/rules

Natural deduction

1. Utility rules

$$\frac{a; b; \dots; c; d \vdash goal}{b; \dots; c; d; a \vdash goal} (\text{shiftL, shiftLeft, left})$$

$$\frac{a; b; \dots; c; d \vdash goal}{d; a; b; \dots; c \vdash goal} (\text{shiftR, shiftRight, right})$$

$$\frac{a_1; \dots; a_n \vdash goal_1 \quad b_1; \dots; b_n \vdash goal_2 \quad \dots \quad c_1; \dots; c_n \vdash goal_n}{b_1; \dots; b_n \vdash goal_2 \quad \dots \quad c_1; \dots; c_n \vdash goal_n \quad a_1; \dots; a_n \vdash goal_1} (\text{next})$$

2. Axiom rules

$$\frac{\dots; a; \dots \vdash a}{\text{Done.}} (\text{axiom, assumption}) \quad \frac{a; b; \dots \vdash c}{b; \dots \vdash c} (\text{weak, destroy})$$

3. Intuitionistical rules

$$\frac{a; \dots \vdash b \wedge c}{a; \dots \vdash b \quad a; \dots \vdash c} (\text{conjI})$$

$$\frac{a \wedge b; \dots \vdash c}{a; b; \dots \vdash c} (\text{conjE})$$

$$\frac{a; \dots \vdash b \vee c}{a; \dots \vdash b} (\text{disjIL}, \text{disjI1})$$

$$\frac{a; \dots \vdash b \vee c}{a; \dots \vdash c} (\text{disjIR}, \text{disjI2})$$

$$\frac{a \vee b; \dots \vdash c}{a; \dots \vdash c \quad b; \dots \vdash c} (\text{disjE})$$

$$\frac{a; \dots \vdash b \Rightarrow c}{a; \dots; b \vdash c} (\text{impI})$$

$$\frac{a \Rightarrow b; \dots \vdash c}{\dots \vdash a \quad b; \dots \vdash c} (\text{impE})$$

$$\frac{a; \dots \vdash b \Leftrightarrow c}{a; \dots \vdash b \Rightarrow c \quad a; \dots \vdash c \Rightarrow b} (\text{iffI})$$

$$\frac{a \Leftrightarrow b; \dots \vdash c}{a \Rightarrow b; b \Rightarrow a; \dots \vdash c} (\text{iffE})$$

$$\frac{a; \dots \vdash \neg b}{a; \dots; b \vdash \text{False}} (\text{notI})$$

$$\frac{\neg a; \dots \vdash b}{\dots \vdash a} (\text{notE})$$

4. Classical rules

$$\frac{a; \dots \vdash b}{a; \dots; \neg b \vdash \text{False}} (\text{ccontra}, \text{ccontr})$$

$$\frac{a; \dots \vdash b}{a; \dots; \neg b \vdash b} (\text{classical})$$

5. Predicate rules

$$\frac{[y_1; y_2; \dots; y_n] \quad q; \dots \vdash \forall x_1 x_2 \dots x_n. (p(x_1, x_2, \dots, x_n) \dots)}{[y_1; y_2; \dots; y_n; x_1] \quad q; \dots \vdash \forall x_2 x_3 \dots x_n. (p(x_1, x_2, \dots, x_n) \dots)} (\text{allI, univI})$$

$$\frac{[y_1; y_2; \dots; y_n] \quad \forall x_1 x_2 \dots x_n. (p(x_1, x_2, \dots, x_n) \dots); \dots \vdash q}{[y_1; y_2; \dots; y_n] \quad \forall x_2 \dots x_n. (p(t, x_2, \dots, x_n) \dots); \dots \vdash q}$$

(allE $x_1 = t$, univE $x_1 = t$)

$$\frac{[y_1; y_2; \dots; y_n] \quad q; \dots \vdash \exists x_1 x_2 \dots x_n. (p(x_1, x_2, \dots, x_n) \dots)}{[y_1, y_2, \dots, y_n] \quad q; \dots \vdash \exists x_2 \dots x_n. (p(y_1, x_2, \dots, x_n) \dots)}$$

(exI $x_1 = y_i$) [$y_i \in \overline{y_1, \dots, y_n}$]

$$\frac{[y_1; y_2; \dots; y_n] \quad \exists x_1 x_2 \dots x_n. (p(x_1, x_2, \dots, x_n) \dots); \dots \vdash q}{[y_1; y_2; \dots; y_n; x_1] \quad \exists x_2 \dots x_n. (p(x_1, x_2, \dots, x_n) \dots); \dots \vdash q} (\text{exE})$$

6. Convenience rules

$$\frac{a; \dots \vdash \neg \neg b}{a; \dots \vdash b} (\text{doubleNegI, notnotI}) \quad \frac{\neg \neg a, \dots \vdash b}{a; \dots \vdash b} (\text{doubleNegE, notnotE})$$

$$\frac{\text{False}; \dots \vdash b}{\text{Done.}} (\text{efq})$$

$$\frac{a; \dots; \neg a; \dots \vdash b}{\text{False}; \dots \vdash b} (\text{contraDef, contrDef}) \quad (\text{coming soon})$$

$$\frac{\forall x_1 \dots x_n. (p(x_1, \dots, x_n) \dots); \dots \vdash q}{\neg(\exists x_1 \dots x_n. (\neg(p(x_1, \dots, x_n) \dots))); \dots \vdash q} (\text{deMorgan, dm})$$

$$\frac{\exists x_1 \dots x_n. (p(x_1, \dots, x_n) \dots); \dots \vdash q}{\neg(\forall x_1 \dots x_n. (\neg(p(x_1, \dots, x_n) \dots))); \dots \vdash q} (\text{deMorgan, dm})$$

Example of a complete proof:

```
lemma |- p => (q => p)
proof
  impI
  .
  axiom
qed
```

Keywords

List of all AIS keywords with its syntactical equivalents:

- *lemma / theorem*
- *lemma: / theorem:*
- *|- / proves*
- *assume*
- *show*
- *proof*
- *qed*
- *next*
- *shiftL / shiftLeft / left*
- *shiftR / shiftRight / right*
- *again / .*
- *axiom / assumption*
- *weak / destroy*
- *conjI*
- *conjE*
- *disjIL / disjI1*
- *disjIR / disjI2*

- *disjE*
- *impl*
- *impE*
- *iffI*
- *iffE*
- *notI*
- *notE*
- *ccontra* / *ccontr*
- *classical*
- *efq*
- *contraDef* / *contrDef*
- *allI* / *univI*
- *allE* / *univE*
- *exI*
- *exE*
- *doubleNegI* / *notnotI*
- *doubleNegE* / *notnotE*
- *deMorgan* / *dm*

Below is the list of all logical constants, connectives and a regex for constructing variables or quantified formulas.

Every logical connective can be represented either by its designated symbol or its keyword!

- True
- False
- *&* / *and*
- *|* / *or*
- *=>* / *implies*
- *<=>* / *iff* / *equiv*
- *=* / *equals*

- $\sim$ / *not* / *neg*
 - $!$ / *univ*
 - $?$ / *exists*
 - P / *possibly* /
 - N / *necessarily* /
 - $\text{term} = ([a-z] \mid [A-M, O, Q-Z]) ([a-z] \mid [A-Z] \mid [0-9])^*$
 - $\text{predicate} = \text{term} \text{ " " "(" " " " term ("," " " " term) " "*" "}"$
 - $\text{quantified_formula} = (! \mid ? \mid P \mid N) (\text{term} \text{ " "})^+ \text{ " (" formula "}"$
-

Closing notes

The text editor comes with a set of example lemmas that can be loaded in immediately. As the nature demands it, the proof of these lemmas is left as an exercise to the user.